Object-Oriented programming
Practice 4

Experimenting with Exception Handling

Getting Ready:

Before going any further, you should download the following files to your working directory.

Example1.java
Example2.java
Example3.java
Example4.java

1. A Simple Example:

This part of the lab considers a simple example of exception handling.

1. Open Example1.java
2. Compile and execute the application Example1.

What was output by the application when you executed it?

The output:

```
The answer is 2. Done
```

3. Change the value of denominator to 0.
4. Re-compile and re-execute Example1.

What "error" was generated by the application when you executed it?

The error:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
at OOP/practice5.Example1.main(Example1.java:10)
```

Why was this "error" generated at run-time (rather than at compile-time)?

Because the code itself is correct, but the arithmetic execution is incorrect

5. Add a try-catch statement. Specifically, put only the statement that generated the exception inside of the try block and put no statements in the catch block. (Hint: You should be able to determine what exception to catch and what line generated the exception from the error message that you received during the previous step.)

Re-compile Example1.

What error is generated and why?

The error:

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
The local variable ratio may not have been initialized
at OOP/practice5.Example1.main(Example1.java:15)
```

The error was generated because compiler doesn't know if the value of the ratio was assigned. If the division throws an exception, ratio wouldn't get a value. Since ratio might be uninitialized, Java won't compile the code.

6. Move the "output statement" into the try block (as well).
7. Add the statement System.out.println("Divide by 0."); to the catch block.

Re-compile and re-execute Example1.

What output was generated?

The output:

```
Divide by 0. Done.
```

8. Add a call to the printStackTrace() method of the ArithmeticException to the end of the catch block.

Re-compile and re-execute Example1.

What output was generated?

The output:

```
Divide by 0.
java.lang.ArithmeticException: / by zero
at OOP/practice5.Example1.main(Example1.java:10)
Done.
```

Did the application execute properly or not?

Yes. The application executed and printed error that was catched

2. A More Complicated Example:

This part of the lab considers an example of exception handling within and outside of block statements.

- Open Example2.java
- Compile Example2.

What error was generated?

The error:

```
Exception in thread "main" java.lang.Error: Unresolved compilation problems:
The local variable i may not have been initialized
The local variable i may not have been initialized

at OOP/practice5.Example2.main(Example2.java:17)
```

- Initialize i to 0 inside of the try block (but before the for loop).
- Compile Example2.

What error was generated?

The error:

```
Exception in thread "main" java.lang.Error: Unresolved compilation problems:
i cannot be resolved to a variable
i cannot be resolved to a variable

at OOP/practice5.Example2.main(Example2.java:18)
```

- It is not possible for i to be used before it is initialized. Why is this error generated anyway? (Hint: Think about block statements.)

  The error was generated because i was initialized inside "try" block and not reachable inside "catch" block

- Move the initialization of i before the try block.

- Compile and execute Example2.

What output is generated?

The output:

```
100/10=10
Couldn't calculate 10/0
```

Why aren't all of the divisions even attempted?

After an exception occurs, the program immediately moves to the "catch" block and skips remained parts of the "try" block

- Fix Example2 so that it executes properly. (Hint: Move the try-catch block inside of the for block.) What did you change? What has happened?

  I've moved try-catch construction inside for loop, so it will proceed all the iteration and for each try-catch will be executed. If one division fails, it will only affects that iteration, not the rest.

3. An Inappropriate Use of Exception Handling

This part of the lab considers an inappropriate use of exception handling and how to "fix" it.

- Compile and execute Example3 and verify that it outputs all of the values followed by the word "Done".

- Modify Example3 so that it loops "properly" and does not need to use a try-catch statement. (Note: The output should not change.) What did you change?

  It can be easily fixed by providing correct boundary – I've just changed right bound from "10" to data.length

4. Some Other Exceptions

This part of the lab will give you some experience with some other exceptions, where they arise, and how they can be used.

- **What functionality does a StringTokenizer object provide? Give example.**

  The answer: StringTokenizer is a class that is used to break a string into tokens using delimiters.

  Example:

```
String str = "Hello,world";
StringTokenizer st = new StringTokenizer(str, ",");
while(st.hasMoreTokens()){
  String token = st.nextToken();
  System.out.println(token);
}
```

- **What are the three formal parameters of the explicit value constructor in the StringTokenizer class? Give example.**

  The answer: StringTokenizer(String str, String delim, boolean returnDelims)

str – the string to be tokenized
delim – a string containing delimiter characters
returnDelims – should delimiters be considered and returned as tokens

- Run Example4.java.

- After running the program write the following to the command line: 5.3+9.2

What output is generated?

The output:

```
14.5
```

- Now run it again and enter the following: 5.3+

What output is generated?

The output:

```
Invalid syntax
```

Why? In particular, what exception is thrown and why?

The answer: The program expects 3 tokens, while input provided only 2 tokens. Calling nextToken() throws NoSuchElementException, that is handled as "Invalid syntax" message

- Run again and enter 5.3+a.

What output is generated?

The output:

```
One or more operands is not a number
```

Why? In particular, what exception is thrown and why?

The answer: The program expects operands to be numbers, while input provided character. Calling Double.parseDouble() throws NumberFormatException, that is handled as "One or more operands is not a number"

5. Programming Practice

- Modify Example4.java so that it supports addition (+), subtraction (-), multiplication (*), and division (/).

  ✓

- Modify Example4.java so that it processes more than just one expression for evaluation. So, for example, it should be able to be execute the following input:

  45.0+4.1 3.2*9.1.

  ✓

- Modify Example4.java so that it tells you which operand is not a number. (Hint: You may need to use nested try-catch blocks.)

  ✓